
MessageFoundry — Early-Adopter Installation & Rollout Guide

Early Access · MessageFoundry v0.3.2 · September 2026

Use this guide to pilot **MessageFoundry (MEFOR)** and plan a staged rollout. It links installation, security, testing, recovery, and operations references into one process.

Read this section first. MessageFoundry started in May 2026 and remains **Early Access, beta-level software** under fast development. Start with synthetic data in a sandbox. The stages in §11 help you find defects before production cutover. Review the limits in §2 and advance only after each stage's checks pass.

Table of contents

1. What MessageFoundry is, and who should pilot it
 2. Maturity & honest limitations — read before you plan
 3. Prerequisites & environment checklist
 4. Installation
 5. Minimum viable configuration
 6. Security & PHI hardening before real data
 7. Reliability configuration — how nothing gets lost
 8. Pre-traffic validation
 9. Capacity & load testing on *your* hardware
 10. Backup, restore & disaster recovery
 11. Staged rollout plan with go/no-go gates
 12. Day-2 operations & monitoring
 13. Upgrade & rollback
 14. High availability — rollout & VIP standup runbook
 15. Getting help & reporting bugs
 16. Decommissioning a pilot
-

1. What MessageFoundry is, and who should pilot it

MessageFoundry routes, transforms, and validates messages between Connections. HL7 v2.x is the default, with payload-agnostic support for other formats. Routing and handling use Python. The runtime model is a graph wired by name:

- **Connection** — an endpoint that receives (inbound) or sends (outbound) messages (MLLP, TCP, File today. REST/SOAP/Database destinations and a Database poll source also ship — see [CONNECTIONS.md](#)).
- **Router** — a Python function bound to an inbound connection that decides which Handler(s) see each message.
- **Handler** — a Python function that filters → transforms a message and emits **Send s** to outbound connections.

The headless asyncio service (FastAPI/uvicorn) owns the durable message store. It supervises one worker set per connection. A browser **web console** ([/ui](#)) and a **VS Code extension** operate it over a localhost HTTP API. See [ARCHITECTURE.md](#) for the full model.

Pilot MessageFoundry if your team can test a pre-1.0 Python engine against its own needs. Start with one node on a trusted network. Native TLS, local-account TOTP MFA, and off-box log/audit forwarding are available. Optional active-passive failover uses PostgreSQL or SQL Server (\$2/\$6/\$14). Review the de-identification limit in §2. Active-active scale-out was dropped. Active-passive is the supported high-availability model.

2. Maturity & honest limitations — read before you plan

MessageFoundry is a new, beta-level project. It supports single-node deployment and optional **active-passive failover** on PostgreSQL or SQL Server (\$14). Consult [ARCHITECTURE.md](#) and the README Roadmap alongside the technical limits below. Validate your intended deployment before relying on it.

Included capabilities

Capability	Status
Code-first Connection/Router/Handler graph	✔ Built
SQLite (WAL) store backend	Supported. The default for single-node and development installations.
PostgreSQL store backend (single-node)	Supports the staged pipeline, at-rest encryption, and retention. Single-node behavior matches SQLite.
Microsoft SQL Server store backend (single-node)	Supports the staged pipeline, response capture, and at-rest encryption. Requires the <code>sqlserver</code> extra and OS-level ODBC Driver 18. Engine retention and purge match SQLite. The database administrator owns WAL checkpoint, <code>VACUUM</code> , and database-level <code>.mfbak</code> snapshots.
Transactional staged queue (ingress→routed→outbound), at-least-once, dead-letter, replay	✔ Built — see ADR 0001

Capability	Status
Auth + RBAC + hash-chained audit log	✓ Built — see SECURITY.md
At-rest body encryption (AES-256-GCM, opt-in) + key rotation	✓ Built — see PHI.md
MLLP / TCP / File connectors. REST / SOAP / Database destinations. Database poll source	✓ Built — see CONNECTIONS.md
Validation & load tooling (<code>generate</code> , <code>check</code> , <code>dryrun</code> , the test harness, the load harness)	✓ Built — see §8/§9 and LOAD-TESTING.md
Windows-service deployment via NSSM	✓ Built — see SERVICE.md
Native transport TLS (API + MLLP)	Supports API HTTPS/WSS and per-connection MLLP-over-TLS, with minimum TLS 1.2 and optional mTLS. Startup rejects off-loopback binds without TLS. Raw TCP/X12 remain plaintext and require loopback or a proxy. See DEPLOYMENT.md .
Native MFA (TOTP, local accounts)	Supports RFC 6238 TOTP and single-use recovery codes. <code>[security].require_mfa</code> defaults on. <code>require_mfa_scope</code> defaults to <code>every_local_account</code> . Set <code>administrators</code> for the narrower scope. AD/Entra MFA remains the identity provider's responsibility. See SECURITY.md .
Off-box log + audit forwarding	<code>[logging].forward_*</code> sends operational logs and PHI-redacted audit rows to syslog/SIEM. Set <code>forward_protocol = "tls"</code> for RFC 5425 transport on port 6514 (ADR 0080). Set the CA anchor with <code>forward_tls_*</code> . The default transport is UDP. Select TLS or use a local TLS-forwarding agent. See PHI.md §7.
Active-passive HA / failover	Optional leader/standby cluster on shared PostgreSQL or SQL Server storage (Track B). Only the leader runs the graph. A leadership lease enforces self-fencing, with recovery on promotion. Single-node remains the unchanged default. See CLUSTERING.md and §14.

Experimental or not yet built — do not depend on these for a production pilot

Capability	Status & implication
Transport TLS for raw TCP / X12	✗ Not built — those two connectors are plaintext-only. Keep them on loopback or front with a TLS-terminating proxy. (API + MLLP do have native TLS — see §6/ DEPLOYMENT.md .)
<code>ack_after=delivered</code> (defer the ACK until downstream delivery)	Not implemented. Configuration load rejects this setting. Only ACK-on-receipt is supported. Routing, transform, and delivery failures occur after <code>AA</code> and do not return a NAK. Operators must monitor message status and alerts.
De-identification framework	✗ Not built. The AI assistant's <code>deidentified</code> scope falls back to <code>code_only</code> .
In-place SQLite → server-DB migration	✗ Not built. Server-DB deployments are greenfield only — there is no automatic carry-over of SQLite history. Drain and cut over deliberately (§13).

Capability	Status & implication
A throughput guarantee for your hardware	⚠ By design. The published baseline and tuning method (TUNING-BASELINE.md, Gate #3) separates conformance from performance. Conformance checks are host-independent requirements. Performance results are "as measured on the reference config". Because the durable-write path is hardware-dependent, those msg/s are not a promise for your box. Measure on your own hardware (§9).

Before adopting, measure capacity on your hardware and address the limits above, including de-identification. Use the staged rollout below to test delivery, security settings, and recovery.

3. Prerequisites & environment checklist

Confirm these prerequisites before installing:

- [] **Python 3.14+** on the engine host.
- [] **OS:** Windows is the primary supported/serviced platform (NSSM). The engine itself is cross-platform Python.
- [] **Administrator/elevation** on the host if you will install the Windows service.
- [] **Outbound network access** for the service installer to download the SHA-256-pinned NSSM binary (or pre-stage NSSM on the host / on `PATH`).
- [] **Firewall plan:** open your inbound MLLP listener port(s) (the samples use e.g. `2575 / 2600`) to senders, and decide who may reach the **API on** `127.0.0.1:8765` (default loopback — keep it that way. See §6).
- [] **A writable data directory** for the store + logs (service default: `C:\ProgramData\MessageFoundry`).
- [] **Backend decision (made here, not later):** Use SQLite (default, zero extra dependencies) for a single-node pilot. For a server store or database-tier HA, select PostgreSQL or SQL Server. PostgreSQL uses `messagefoundry[postgres]` (pure-Python). SQL Server uses `messagefoundry[sqlserver]` and OS-level ODBC Driver 18. See §2.
- [] If you will run the **cluster** path (active-passive failover): **NTP time sync** across nodes is a hard prerequisite, every node needs the **same config dir**, and `[store].backend = "postgres" or "sqlserver"`. (Single-node pilots skip this entirely. See §14.)
- [] A **PHI encryption key** plan (§6) and a **backup target + key-escrow** plan (§10) decided before any real data flows.

4. Installation

Read the [Installation Guide](#). It explains engine installation, a private configuration repository, and multiple instances from one repository. This section is the rollout-oriented summary of the same material.

Full reference: the [Installation Guide](#) and [SERVICE.md](#). The essentials:

4.1 Install the engine

MessageFoundry is a **read-only, version-pinned dependency** (ADR 0017): install a published wheel and **pin the exact version**, the same way you pin any other production dependency. Create a venv and install:

```
python -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install "messagefoundry==0.3.2"          # pin the exact engine version (core runtime only)
```

`messagefoundry==0.3.2` pulls only the **core runtime** — what a headless engine needs. Add extras (\$4.2) for the PySide6 test harness, a server-DB backend, or SFTP. The browser web console installs as its own `messagefoundry-webconsole` wheel.

⚠ Early Access. MessageFoundry is beta-level software on public PyPI. The independent external review and penetration test recommended by ASVS Level 3 have not occurred. Pin the version that your organization has tested. Check [PyPI](#) for the current release. You can also install from your organization's private index.

Verify each release before installation. A version or hash identifies a fixed file but does not identify its builder. Each release includes **SLSA build provenance** and a **Sigstore signature**. Use the **GitHub CLI** (`gh` ≥ 2.49) and optionally `sigstore` (`pip install sigstore`) to check them. Install only the verified file:

```
$V = "0.3.2"    # the version you intend to install – see PyPI for the current release

# Download the wheel + its Sigstore bundle from that release's assets
gh release download "v$V" --repo MEFORORG/MessageFoundry `
  --pattern "messagefoundry-$V-*.whl" --pattern "messagefoundry-$V-*.whl.sigstore*"

# Verify SLSA build provenance: artifact -> source commit -> builder workflow
gh attestation verify "messagefoundry-$V-py3-none-any.whl" --repo MEFORORG/MessageFoundry

# (defense in depth) Verify the Sigstore signature pins the release workflow identity
python -m sigstore verify identity "messagefoundry-$V-py3-none-any.whl" `
  --cert-identity "https://github.com/MEFORORG/MessageFoundry/.github/workflows/release.yml@refs/tags/v$V" `
  --cert-oidc-issuer "https://token.actions.githubusercontent.com"

# Only if BOTH pass, install the exact file you verified
pip install ".\messagefoundry-$V-py3-none-any.whl"
```

The same attestation also covers the **public PyPI** copy (it is byte-identical), so you can `pip download "messagefoundry==$V" --no-deps -d .\verify`, `gh attestation verify` the downloaded wheel, then `pip install --no-index --find-links .\verify "messagefoundry==$V"`. A registry/mirror substitution or a relabelled file **fails** the check. (The `--cert-identity` ref must match the tag you install — e.g. `refs/tags/v0.3.2` for that release.)

For repeatable deployment, generate a hash-locked requirements file for the installed extras. Install it with `--require-hashes`. The generated configuration repository (`messagefoundry init`, below) pins the engine in `requirements.txt`. Extend that file into a full hash-lock.

4.2 Optional extras

Extra	Pulls in	When
<code>postgres</code>	<code>asyncpg</code> (no OS dep. Ships compiled wheels)	Using the PostgreSQL backend (recommended prod path)
<code>harness</code>	PySide6	Running the standalone test-harness GUI
<code>sftp</code>	paramiko	SFTP connectors
<code>sqlserver</code>	<code>aiodbc</code> + OS-level Microsoft ODBC Driver 18	The SQL Server <i>store</i> backend (<code>backend=sqlserver</code> , production) and the DATABASE connector family.
<code>dev</code>	pytest (+ <code>asyncio/timeout/rerunfailures</code> plugins), <code>ruff</code> , <code>mypy</code>	Development & CI

⚠ `messagefoundry check` cannot catch a missing `postgres` extra — it validates a *declared* backend without dialing the database. If `backend=postgres` lacks its extra, startup fails with an error that names the required package and installation command. Install it with `pip install 'messagefoundry[postgres]'`. The `sqlserver` backend has the same requirement. Install the extra with the backend.

Start your own config repo (`messagefoundry init`)

After installing the engine, create your separately versioned **configuration repository** (ADR 0017). It holds Connections, Routers, and Handlers for one or more instances. The installed wheel does not include `samples/`. Those files require a source checkout. Use your own `config/` directory with a wheel install.

```
messagefoundry init ./my-config-repo
```

The command creates a starter feed in `config/`, environment files in `environments/<env>.toml`, and a synthetic fixture. It also creates `messagefoundry.toml`, a pinned `requirements.txt`, a continuous integration `check` workflow, and `.vscode` settings. Run `messagefoundry check --config config --messages messages/sets` to validate the initial configuration. Read the generated `README.md` for daily instructions.

4.3 Run it (foreground, to learn the ropes)

From your config repo (the one `messagefoundry init` created), run the engine against its `config/` directory:

```
cd ./my-config-repo
python -m messagefoundry serve --config config --db ./messagefoundry.db --env dev
```

`serve` flags and their precedence (**CLI** > **MEFOR_<SECTION>_<KEY>** **env** > **messagefoundry.toml** > **built-in default**): `--config` (your graph directory — pass `--config config` for a scaffolded repo. the built-in default `samples/config` exists only in a source checkout), `--service-config` (default `./messagefoundry.toml` if present), `--db`, `--host`, `--port`, `--log-level`, `--env` (a **free-form** environment name, ADR 0017), `--allow-insecure-bind`.

⚠ **The active environment is required.** Without `--env <name>` or `[ai].environment`, `serve` exits 2. It has no default `prod` environment. Missing settings cannot select another environment's values or secrets. Built-in names `dev` / `staging` / `prod` carry a default posture. a custom name (e.g. `test`, `poc`) also needs `[security].handles_real_patient_data` + `[security].production_instance` (both booleans). The active environment is logged at startup.

🔑 **dev now carries the PHI posture (ADR 0148) — provide a store key or declare synthetic.** ADR 0148 (GIVEN 1) gives the built-in `dev` environment the PHI data class. The first run therefore tests the production at-rest encryption path. `serve --env dev` therefore **refuses to start (exit 2) without a store encryption key**. Two ways forward for a local run: - **Recommended — mint a throwaway dev key** (exercises the real encryption path): run `messagefoundry gen-key` and set the printed base64 value as `MEFOR_STORE_ENCRYPTION_KEY` (a dev key is fine. **never commit it**). - **Genuinely no-PHI box** — declare it synthetic: set `[security].handles_real_patient_data = false` (a loud, audited opt-out) to run **key-free**, for a dev/CI box that only ever processes synthetic HL7.

The refuse/warn severity of the PHI serve-gate ladder is the `[security].enforcement` dial (default `enforce`, byte-identical to the former production behaviour). A loopback development bind checks the store-key requirement above. TLS and MFA exposure checks apply only to non-loopback binds. Set `[security].enforcement = warn` to downgrade the PHI refusals to loud, audited warnings during local bring-up.

4.4 Run it as a Windows service (the supported production run-mode)

Use the elevated installer. It already defaults to a **least-privilege per-service virtual account** (`NT SERVICE\<ServiceName>`, no password). Pass `-ServiceAccount` only to run under a *different* account, and `-AllowLocalSystem` to opt out to LocalSystem. `-Environment` is **required** (ADR 0017) — the installer refuses without it rather than registering a service that dies on every start:

```
# from an elevated shell
scripts\service\install-service.ps1 -Environment prod
```

The installer can reconfigure an existing service. It downloads SHA-256-pinned NSSM and stores absolute `serve` paths. With `-ServiceAccount`, it grants configuration-read and data-directory read/write access. Service defaults: name `MessageFoundry`, data dir `C:\ProgramData\MessageFoundry`, store `<DataDir>\messagefoundry.db`, logs `<DataDir>\logs`, bind `127.0.0.1:8765`.

⚠ **Pinned-wheel operational model.** With a pinned-version install (§4.1), the service loads the installed wheel at that fixed version. To upgrade, run `pip install "messagefoundry==<new>"`. Then restart NSSM (§13). Review each version change. *(A contributor running the editable install instead serves whatever branch is checked out — treat that checkout as the release artifact. See §13.)*

4.5 First-run admin bootstrap

Auth is **enabled by default**. On first startup with an empty store, MEFOR creates the bootstrap administrator (`admin`). It writes the password to owner-only `bootstrap-admin.txt` beside the store. It logs only the file location, never the password. Then:

1. Log in as `admin`. You are **forced to change the password** on first use.
2. **Create a second real administrator** promptly.
3. **Delete** `bootstrap-admin.txt`.

The bootstrap account auto-retires once a second admin exists, or — while still unclaimed — 72h after creation.

4.6 Verify it runs

```
curl http://127.0.0.1:8765/health      # -> {"status":"ok"}
# tail <DataDir>\logs\service.out.log for the "wiring started" banner
```

Then send a synthetic message to confirm the end-to-end path. The starter feed listens on MLLP `2575`. It includes a PHI-free fixture at `messages/sets/example_adt.h17`. Send this fixture with an MLLP client. The convenience senders (`samples/send_mllp.py`, `python -m harness`) ship with the **engine source checkout**, not the installed wheel. From a checkout you can run:

```
python samples/send_mllp.py samples/messages/adt_a01.h17
```

If startup fails, first check `service.err.log`. Common causes include relative paths that resolve to the system directory, port conflicts, and insufficient data-directory write access.

5. Minimum viable configuration

Full reference: **CONFIGURATION.md** (service settings) and **CONNECTIONS.md** (the graph).

There are two distinct configuration surfaces:

1. **The message graph (Python modules)** in your `--config` directory. The minimum first flow is one module: The module defines an `inbound()` with a transport specification and `router=` binding. A `@router` selects handler names. A `@handler` returns `Send(...)` to a declared `outbound()`. The scaffolded repo

(`messagefoundry init`, §4) gives you a working `config/IB_EXAMPLE_ADT.py` to start from (or, from a source checkout, copy `samples/config/IB_ACME_ADT.py`). The loader globs `*.py` (non-recursive. skips `_*`-prefixed helper files), then merges an optional `connections.toml`.

2. **Service/operational settings** in `messagefoundry.toml` (+ `MEFOR_*` env + CLI). Keep **all secrets out of this file and out of source control** — supply them via `MEFOR_<SECTION>_<KEY>` env vars (the loader *warns* if it sees a known secret in the file).

Guidance for a clean first flow:

- **Use the MLLP/File pair** for an initial end-to-end test — both are fully built and need no extras. The Database connector family is production-supported but adds the `[sqlserver]` extra + ODBC Driver 18. MLLP/File keep the first hop dependency-free.
- **Never set a host on an inbound MLLP/TCP connection** (it is a config error). Set the listen interface once, service-side, via `[inbound].bind_host` (loopback for dev. A specific NIC behind a firewall for prod). Outbound MLLP/TCP *do* take the downstream host.
- Use `env("key")` for environment-specific values. Keep non-secret values in `environments/dev.toml` and `environments/prod.toml` with identical keys. Supply secrets only through `MEFOR_VALUE_<KEY>`. A referenced-but-undefined key fails loud at load. Use `current_environment()` (not `env()`) inside a handler to branch on the deployment.
- **`connections.toml` (data) is optional** (ADR 0007): move *transport config* there if you want hand/GUI editing. Keep *logic* (routers/handlers) in `.py`. A name declared in both a module and `connections.toml` is a hard error (no silent shadowing).
- **The `--config` directory is a trust boundary.** `serve` and `POST /config/reload` **execute** the Python in it, in-process, as the service account. On POSIX the loader refuses a group/world-writable config dir. **on Windows this is your responsibility** — lock the directory's ACL to admins + the service account.

6. Security & PHI hardening before real data

Full references: **SECURITY.md**, **PHI.md**, and **DEPLOYMENT.md** (network exposure). MEFOR provides authentication, RBAC, audit, and optional at-rest encryption. It supports native TLS (API + MLLP, with off-loopback guards), local-account TOTP MFA, and remote log/audit forwarding. The remaining transport gap is **TLS for the raw TCP and X12 connectors**, which are plaintext-only. Complete this checklist **before any real PHI flows**:

- [] **API off-loopback requires native TLS.** The API binds `127.0.0.1` by default. To reach it from another host, configure **in-process TLS** (`[api].tls_cert_file` + `[api].tls_key_file`, `tls_min_version` $\geq$ 1.2, opt-in mTLS via `tls_client_ca_file`) **or** front it with a TLS terminator (`[api].tls_terminated_upstream = true` + `[api].trusted_proxies`). A non-loopback bind **without** TLS (or a trusted terminator) is **refused at startup**. Never use `--allow-insecure-bind` with real PHI. This development-only option can expose bearer tokens and PHI as cleartext. (With auth disabled, a non-loopback bind is refused unconditionally.)

- [] **MLLP off-loopback requires native TLS too.** MLLP-over-TLS is built: set `tls = true` + `tls_cert_file` / `tls_key_file` per connection (opt-in mTLS via `tls_ca_file` . ≥ TLS 1.2). MLLP is **plaintext by default**, and a non-loopback plaintext MLLP bind is refused. **Raw TCP and X12 have no transport TLS** — keep them on a trusted segment or proxy-terminate. Full matrix: [DEPLOYMENT.md](#).
- [] **Turn on at-rest encryption and make it mandatory:** mint a key with `messagefoundry gen-key` (or a Windows DPAPI-protected key file via `messagefoundry protect-key`), set `MEFOR_STORE_ENCRYPTION_KEY` , and set `[store].require_encryption = true` so the engine refuses to start unencrypted.
- [] **Enable volume encryption (BitLocker/LUKS).** App-level encryption protects message *bodies*. the `summary` / `control_id` / `message_type` columns and the `-wal` / `-shm` /temp files are **not** app-encrypted and rely on volume encryption.
- [] **Run under a least-privilege account** (the virtual account from §4.4) and lock down the store directory and any File-connector spill directories. **Treat backups as PHI.**
- [] **Finish the bootstrap-admin handoff** (§4.5): change the password, create a second admin, delete `bootstrap-admin.txt` .
- [] **For Active Directory:** use **LDAPS** with a trusted CA, never set `MEFOR_ALLOW_INSECURE_TLS` in production, and configure the directory's lockout/complexity policy (the engine's account lockout covers local accounts only). AD/Entra MFA is enforced by your directory. Local accounts use native TOTP MFA (`[security].require_mfa` , WP-14). It defaults on for every local account. Keep it enabled before off-loopback PHI exposure.
- [] **Populate the fail-closed `[egress]` allowlist** (it defaults to unrestricted) for REST/Database destinations.
- [] **Keep logging at `INFO` or above** and `expose_docs` off in production. The engine does not log full payloads at INFO+. PHI redaction of chained-exception traceback text remains incomplete. Do not use `DEBUG` with real PHI.
- [] **Author routers/handlers so they never interpolate raw HL7 into an exception message** (it can surface in `last_error` / `detail`).
- [] Run `messagefoundry audit-verify` periodically (the audit log is *tamper-evident*, not *tamper-proof*), and set `[retention]` windows — they are **off by default (kept forever)**.

7. Reliability configuration — how nothing gets lost

The engine uses a **transactional staged queue** with ingress, routed, and outbound stages. Each handoff commits in one transaction, supporting **at-least-once** delivery and reruns after crashes. It needs no external broker. See [ADR 0001](#).

Understand these delivery rules:

- **ACK-on-receipt.** The sender is `AA` 'd as soon as the raw message is durably committed (after synchronous decode/parse/optional strict-validate, which still NAK). **Any routing/transform/delivery failure happens *after* the ACK** and surfaces as an internal **disposition + alert**, never a NAK. Operators monitor disposition + alerts, **not** the ACK, for post-ingress failures.

- **Disposition lifecycle:** `RECEIVED` → `ROUTED / UNROUTED` → `PROCESSED / FILTERED / ERROR`. The store finalizer is the **sole authority** and never finalizes while any stage row is still in flight. Note: A dead row at any stage sets the entire message to `ERROR`, even if another Handler delivered it. Read the per-message event trail.
- **Failure classification & policy (per outbound):**
 - Permanent partner reject (`AR / CR`) → **dead-letter immediately** (still replayable).
 - Transient (`AE / CE`) or transport error → **retry per `RetryPolicy`**.
 - Internal/code error → either **STOP** the lane and raise a `connection_stopped` alert, or **CONTINUE** (auto-dead-letter the bad message and keep flowing).
 - `RetryPolicy.max_attempts` **unset = retry forever** (nothing silently lost) with exponential backoff. Under the default **FIFO** ordering, a permanently-failing head **blocks its lane** until it succeeds or is purged.

Mandatory before go-live:

- [] **Wire real alerts.** Configure the `[alerts]` **webhook and/or email** notifier — do **not** rely on the default logging-only sink. The conservative defaults (FIFO head-of-line blocking, retry-forever, STOP-on-internal-error) are only safe if a human gets paged when a lane stalls.
- [] **Set `[delivery]` buildup thresholds** (`buildup_max_oldest_seconds` defaults to 300s. Set a `buildup_max_depth` sized to each connection's throughput) so `queue_buildup` fires before a stuck lane silently backs up. Buildup detection now covers the ingress and routed stages too, not just outbound.
- [] **Choose `RetryPolicy` per outbound deliberately:** Select retry-forever when the partner must receive every message. This choice can block the queue and requires buildup alerts. Select finite `max_attempts` when stale data is worse than a replayable dead letter.
- [] **Choose `InternalErrorPolicy` intentionally:** `CONTINUE` (default) for high-volume feeds where uptime matters most. `STOP` for low-volume feeds where ordering/no-loss matters more than uptime.
- [] **Code routers/handlers as pure and idempotent.** At-least-once means a message can re-run after a crash or a replay. No side-effecting writes mid-transform. The **one** allowed exception is a **live, read-only DB lookup**. Downstream connectors/partners must **dedupe** (e.g. On MSH control id).

Use `/dead-letters` to inspect failures and `/dead-letters/replay` for bulk outbound recovery. Use `/messages/{id}/replay` for dead ingress or routed rows, including transform errors, undecryptable messages, and removed handlers. Startup returns stale in-flight rows to pending. It dead-letters rows whose destination or handler is absent from configuration.

8. Pre-traffic validation

Validate before sending network traffic. Use synthetic data only: `generate` and `dryrun` can print full message bodies. Never redirect that output to a committed file or continuous integration log.

1. **Build a synthetic corpus:** `messagefoundry generate --type ADT --count 50 --out <fixtures>` (conformant HL7 v2.5.1, validated against hl7apy. 13 message types, 57 ADT triggers. PHI-free).

2. **Gate the config in CI / a pre-commit hook:** `messagefoundry check --config <dir> --messages <fixtures>`.
 - `validate` (every module loads. Every inbound→router reference resolves. No port collisions) is **required and blocking**. - `dryrun` requires a fixtures directory with `*.h17` files. Without fixtures, the gate skips `dryrun` and uses `validate` alone. It then cannot detect transform runtime errors. **Build and maintain the fixtures.** - `ruff / mypy` are advisory (never block).
3. **Inspect the wiring:** `messagefoundry validate --json` (all problems at once) and `messagefoundry graph --config <dir>` (confirm the wired graph matches intent).
4. **Confirm dispositions:** `messagefoundry dryrun` runs the same core the live engine runs (no I/O), so dry-run and live route identically. Then exercise the **test harness** (`harness/`): Its 5 headless `--scenario` runs (`processed / filtered / unrouted / error / dead_letter`) check results through the API for continuous integration. The GUI injects delivery faults (delay-then-AA, close, fail-N-then-AA). Use these faults to test retries, dead letters, and replay.

Note: `validate` only catches **literal** port collisions. `env()`-resolved ports are checked at bind time. A `prod`-only missing `env()` value may not surface during a `dev`-context check — validate against the target environment before promoting.

9. Capacity & load testing on *your* hardware

Full references: **LOAD-TESTING.md** and the published **throughput baseline & tuning reference** (Gate #3) — a **two-tier gate**: host-independent **conformance** invariants (zero loss, bounded drain, low error rate — a hard release blocker) plus **performance** numbers "*as measured on the reference config*". Durable-write performance depends on hardware. These msg/s figures do not guarantee performance on your host. Measure your own baseline.

The headless load harness (`harness/load/`) uses MLLP and the HTTP API to test a running engine. It never accesses the store. Change the engine's `--db` to compare SQLite and Postgres with identical traffic.

Recommended ramp:

1. **smoke** — tiny zero-loss wiring check (no performance claim).
2. **fanout-baseline** — warmup → ramp → sustained → spike → recovery. SLOs are evaluated only on the measured sustained phases. Reference targets in the profile: ≥ 200 msg/s sustained, ACK p99 ≤ 50 ms, e2e p99 ≤ 5 s, error ≤ 0.001 , drain ≤ 60 s, zero-loss.
3. **soak** — ~1-hour steady state watching DB/WAL growth + dead-letter accumulation.

First, check **zero-loss reconciliation**: `sent == engine_read`, `sink_received == engine_written`, and zero remaining backlog.

Use a closed-loop phase with fixed concurrency to measure sustainable throughput. Use `slow` transform mode to measure the per-core limit. Save the JSON/CSV reports. Use `--baseline` and `--tolerance` to detect regressions.

Set `correlator_capacity` above peak in-flight volume. Check for correlation-miss messages. If one Python sender cannot saturate the engine, divide sender work across processes.

Sizing reality: the staged pipeline has $\sim 3\times$ **write-amplification** on SQLite (3 commits for a common single-handler message. 2 + H for an H-way fan-out) — see [the write-amplification benchmark](#). Reserve disk space for `.db` and `-wal`. Configure retention and VACUUM (§10). If SQLite's single writer limits throughput, use Postgres.

10. Backup, restore & disaster recovery

Rehearse a full restore before carrying real data. Your organization owns the backup and recovery process.

Back up the store.

- **SQLite:** the WAL backend means three files must be captured **consistently** — `.db`, `.db-wal`, `.db-shm`. Use `sqlite3 <db> ".backup '<dest>'"` against the live DB, or take a **quiesced cold copy** (graceful stop → copy → restart). A naive copy of just the `.db` while the service runs can be inconsistent.
- **PostgreSQL:** use your standard DB backups — `pg_dump` for logical backups and/or WAL archiving / PITR for point-in-time recovery. The engine is greenfield-only on server DBs, so the DB tier owns store-level DR here.

Escrow the encryption key SEPARATELY. If you enabled at-rest encryption (§6), a restored store is **unreadable without the same** `MEFOR_STORE_ENCRYPTION_KEY` / **DPAPI key file**. Back the key up in a different location/system from the data, with its own access control.

Restore-and-verify drill (do this in the lab, §11 Stage 0): Restore the store and key on a clean host. Start the engine. Check `/health`. Run `/status/integrity-check` (SQLite `PRAGMA quick_check`). Inspect selected `/messages` records and their results.

Keep the store bounded. `[retention]` is **off by default (kept forever)**. Set `max_db_mb` for the `storage_threshold` alert. Configure `[security].delete_message_bodies_after_days` and `[retention].dead_letter_days` for body removal. Set daily VACUUM to limit store growth and avoid a full disk.

11. Staged rollout plan with go/no-go gates

Follow each rollout stage and meet its **exit criteria** before advancing. Each stage tests a different part of the deployment.

Stage 0 — Lab / standalone

Goal: prove wiring, dispositions, and recovery on a throwaway box, with **synthetic data only**.

Setup: SQLite, loopback, auth on, a synthetic corpus from `messagefoundry generate`.

Exit criteria (→ Stage 1): - [] `messagefoundry check --config <dir> --messages <fixtures>` exits 0 (validate and dryrun green). - [] All 5 disposition `--scenario` runs pass (`processed` / `filtered` / `unrouted` / `error` / `dead_letter`). - [] Complete a retry → dead-letter → replay cycle with

harness fault injection. Confirm that you understand the recovery tools (§7). - [] A **backup + restore** has been rehearsed once (§10).

Stage 1 — Shadow / parallel run

Goal: run MEFOR alongside your **incumbent** engine on **real production traffic** without affecting any downstream system.

How: Duplicate the production inbound feed to a MEFOR instance with a temporary/null output sink. The harness correlation sink is suitable. Alternatively, use a Router that **Send**s only to a dedicated shadow outbound. Compare MEFOR's dispositions and transformed output against the incumbent's outcomes for the same messages.

⚠ **Do not dual-write to real partners in shadow.** At-least-once + non-idempotent downstreams make a true dual-write dangerous. Keep shadow outbounds pointed at a sink unless the partner dedupes.

Exit criteria (→ Stage 2): - [] **Zero-loss reconciliation holds** over a sustained window (e.g. 1–2 weeks) at production volume. - [] MEFOR dispositions/output **match the incumbent's** for the same messages (differences explained). - [] **No unexplained dead-letters.** Every **ERROR** understood. - [] A load test on **production-like hardware** meets your own SLO targets (§9).

Stage 2 — Limited production

Goal: MEFOR becomes the system of record for a **small, low-risk subset** of real feeds (one partner / one low-volume interface).

Prereqs: switch to **Postgres (single-node)** if you need a server DB. **encryption on** (`require_encryption=true`). **alerts wired** and **monitoring in place** (§12). **backups automated. upgrade + rollback runbook validated on staging** (§13).

Exit criteria (→ Stage 3): - [] e2e p99 within your SLO. **zero unexplained dead-letters** over the observation window. - [] **Alert wiring proven by a deliberate fault-injection drill** — you triggered `queue_buildup` / `connection_stopped` and the on-call was actually paged. - [] **Backup + restore rehearsed against the production store** (not just the lab copy). - [] Rollback runbook exercised at least once.

Stage 3 — Full production

Goal: migrate the remaining feeds **in waves** (never big-bang). Keep decommissioning the incumbent as a **separate, later** step so you retain a fallback.

Steady-state expectations: - [] Sustained-load SLO met on production hardware. - [] DR (backup/restore) rehearsed and scheduled. - [] On-call + the failure-drill runbook (§12) in place. - [] If you require HA: Configure the active-passive cluster on shared PostgreSQL or SQL Server storage. Follow §14 for VIP setup and both failover tests. Also configure database-tier HA. Decided and rehearsed before you depend on it.

12. Day-2 operations & monitoring

Verify-it-runs (after every start/restart): `GET /health` → `{"status":"ok"}`, send a synthetic message, and confirm the **"wiring started"** banner in `service.out.log`.

Monitoring surfaces (scrape `/metrics` with Prometheus. Poll the JSON API and watch the logs for the rest):

- `/metrics` — **Prometheus text exposition:** per-connection counters, queue-depth / in-pipeline / oldest-pending gauges, latency histograms, and host CPU/memory. Gated by `monitoring:read` exactly like `/stats`, so point your scraper at it with a service token.
- `/stats` — outbox counts by status.
- `/status` — DB size vs disk free, journal mode, counts. **Scrape db-size-vs-disk-free.**
- `/status/integrity-check` — on-demand store integrity.
- `/ws/stats` — ~1 Hz queue-depth WebSocket.
- `/messages` + `/messages/{id}` — per-message detail and the **event trail** (read this, not just the status — see §7).
- `/dead-letters` — **page on dead-letter accumulation.**
- **AlertSink events** to alert on: `connection_stopped`, `queue_buildup`, `storage_threshold` (wire these to webhook/email — §7).
- `service.err.log` — watch it.

Note: the browser web console (`/ui`) surfaces dispositions, dead-letters, and alerts. The **CLI/API** remain available for scripted dead-letter triage and alert management.

NSSM writes logs under `<DataDir>\logs`. Configure rotation and keep the level at `INFO` or above. `DEBUG` output can expose PHI (§6).

Treat `service.out/err.log` as potential PHI. Restrict file access and include the files in the retention policy. For remote output, use the `[logging].forward_*` syslog/SIEM stream. It applies the same PHI redaction as stdout. Set `forward_protocol = "tls"` because the transport defaults to UDP.

Graceful drain for maintenance: stopping the service (Ctrl+C / NSSM stop) triggers the ASGI lifespan to call `engine.stop()` for a clean drain. Always **drain** → **stop** → **back up** → **change** → **restart** → **verify**.

Failure-drill runbook — rehearse these in the lab/staging before prod:

Symptom	First moves
Stuck FIFO lane (retry-forever head)	<code>queue_buildup</code> alert → inspect <code>/messages</code> for the head → fix-and- <code>replay</code> , or <code>dead_letter_now</code> to unblock the lane (the dead row stays replayable).
Poison message	It dead-letters under <code>CONTINUE</code> (or stops the lane under <code>STOP</code>) → triage via <code>/dead-letters</code> → fix the transform → replay.

Symptom	First moves
Full disk / <code>storage_threshold</code>	Free space / tighten <code>[retention]</code> / <code>VACUUM</code> → confirm <code>/status</code> disk-free recovers.
Crash / unexpected restart	Startup auto-recovers in-flight rows → verify <code>/health</code> , the "wiring started" banner, and that backlog drains.
Planned maintenance	Drain-and-stop (graceful), perform the change, restart, verify.

13. Upgrade & rollback

Safe upgrade runbook (pinned-wheel model): 1. **Drain** inbound (quiesce senders or stop accepting new work) and confirm queues are draining. 2. **Stop** the service (graceful). 3. **Back up** the store **and** the encryption key (§10). 4. **Bump the pinned engine version:** update the pin in your config repo's `requirements.txt` (`messagefoundry==<new>`) and `pip install "messagefoundry==<new>"` into the deployment venv. (A contributor on the editable install instead pulls the target commit/tag and `pip install -e .` — treat that checkout as the release artifact. §4.4.) 5. **Re-validate:** run `messagefoundry check` against your config (and `ruff / mypy / pytest` too if you develop the engine). 6. **Restart** and **verify** (`/health`, "wiring started" banner, `/status`).

Rollback: - **Config rollback** is the cheapest lever: the audited `POST /config/reload` does a quiesce-and-swap to a known-good `--config` directory (confined to the allow-listed reload roots). Keep your last known-good config dir available. - **Engine rollback:** re-pin the prior version (`pip install "messagefoundry==<prev>"`) → restart (same runbook above). (Contributors on the editable install: `git checkout` the prior commit/tag → `reinstall` → `restart`.) - **⚠ Schema/store-level changes are not trivially reversible** against a populated store given the greenfield-only posture (no in-place migration). Plan code/config rollback as your primary path. use **dead-letter replay** to recover messages that a bad transform stranded before the rollback.

Pre-1.0 cadence: pin a released version (`messagefoundry==X.Y.Z`). The **latest release** is the supported target. Before filing an issue, reproduce the problem with the latest release. Make small, frequent upgrades.

14. High availability — rollout & VIP standup runbook

Single-node is the default. Its staged queue provides stored delivery and crash recovery (§7). Use an **active-passive cluster** when you need failover after unattended node loss. Additional standby nodes do not raise throughput. This section covers rollout. See **CLUSTERING.md** for topology, lease behavior, and settings.

Run identical engine processes against **one shared PostgreSQL or SQL Server database** with `[cluster].enabled = true`. One **leader** runs listeners and routing, transform, and delivery workers. Warm standbys send heartbeats and track state until they gain leadership. A shared-database lease makes an isolated leader stop before a standby takes over. Coordination uses the `[store]` connection, with no separate broker or node-to-node socket.

14.1 Stand up the cluster

1. **Pick a server-DB backend and make the DB tier itself HA.** `[store].backend = "postgres" or "sqlserver"` (SQLite is rejected for clustering). MEFOR coordinates the *processing* leader. It does **not** replicate your store — delegate store HA to the database (**PostgreSQL streaming replication / managed-Postgres HA**, or **SQL Server Always On**) and rehearse a DB failover separately.
2. **Provision every node identically.** Same installed engine version, the **same config dir**, and the **same** `[store] target` (server/database/schema) on each host. Set `[store].pool_size ≥ 2` (**≥ 3 recommended** — the leader drives the maintenance loop + reclaim sweep + workers concurrently).
3. **Enable the cluster** in the shared `messagefoundry.toml: toml [cluster] enabled = true heartbeat_seconds = 10.0 # keep the invariant: heartbeat < fence < ttl leader_fence_timeout_seconds = 20.0 # a leader that can't renew self-fences here (split-brain guard) leader_lease_ttl_seconds = 30.0 # a standby may acquire only once the lease has expired` The defaults trade a ~30 s crash-failover for margin. Lower all three **proportionally** for faster failover at the cost of tolerance for a slow DB / GC pause.
4. **Sync clocks (NTP).** Row leases use node wall-clock — keep nodes synced so a lease expiry is not mistimed. (Leadership is evaluated on the *DB* clock, so it is skew-immune. The row leases are not.)
5. **Start the same `serve` command on every node** (run each as its own NSSM service — see [SERVICE.md](#)). Confirm membership: `GET /cluster/nodes` lists every node and names exactly one `leader_node_id`. Each node's `GET /cluster/status` reports `role: "primary"` on one and `"standby"` on the rest.

14.2 Stand up the VIP / load balancer (required)

Only the primary node binds inbound ports. Senders must therefore use a **floating VIP / L4 load balancer** and reconnect after failover.

MEFOR does not supply a load balancer. It supplies the bind behavior and `/cluster/*` endpoints. Operators can use keepalived + IPVS, HAProxy in `tcp` mode, F5, or a cloud L4/network load balancer.

Data plane (MLLP / raw-TCP / X12 inbound) — one VIP per listener port:

- **[] Mode = L4 / TCP pass-through.** These are byte streams — do **not** front them with an HTTP/L7 LB.
- **[] Health check = a plain TCP connect to that exact listener port** on each backend node. Because only the primary binds the port, exactly one backend is ever healthy, so the VIP routes there automatically. On failover the old primary's port closes (goes unhealthy) and the new primary's opens (goes healthy) and the VIP follows. An application-level (MLLP-message) probe is unnecessary — a TCP connect is the correct, sufficient signal.
- **[] Tune the probe to your failover budget.** The VIP repoints in roughly `check_interval × unhealthy_threshold`. Size it well under your acceptable outage and above your DB/GC jitter (a 2–5 s interval is typical).
- **[] If MLLP-over-TLS is on, keep TLS end-to-end** (TCP pass-through) so the engine's own cert is presented. Do not terminate TLS at the VIP unless you re-encrypt to the backends.
- **[] Do not let the LB reap long-lived MLLP sockets** — set TCP idle timeouts generous enough for persistent connections.
- **[] Verify every partner reconnects-on-drop.** On failover, connections to the old primary drop and senders must redial the VIP. This is standard MLLP client behavior — confirm it per partner.

Management plane (console / IDE → engine API) — optional VIP:

- [] The API runs on every node and accesses the shared database. An API VIP can use unauthenticated `GET /health` for liveness checks. You can point the console at any node.
- [] To pin operators to the active leader, read `GET /cluster/status` → `role` (`primary` / `standby`) or `GET /cluster/nodes` → `leader_node_id` / `lease_owner` (the console already surfaces the live primary). Both endpoints need only `MONITORING_READ` (VIEWER and up. No PHI).

14.3 Rehearse & validate failover (before real traffic)

Failover is **not instantaneous** — quantify *your* window from these drills. Do not assume zero-downtime.

- [] **Clean switchover** (planned): gracefully stop the primary's service. It **expires its lease**, so a standby promotes on its next heartbeat ($\approx$ `one heartbeat_seconds`). Watch `leader_node_id` move on `/cluster/nodes`, watch the VIP repoint, and keep synthetic traffic flowing throughout.
- [] **Crash** (unplanned): hard-kill / power off the primary. Its lease **ages out**, so a standby promotes after **up to** `leader_lease_ttl_seconds` (~30 s default). A partitioned old primary **self-fences within** `leader_fence_timeout_seconds` so it cannot double-process. Confirm the new primary's **owner-scoped on-promotion recovery** re-pends the dead primary's in-flight rows immediately (delivery resumes without waiting out the per-row lease TTL).
- [] **Zero-loss across the failover**. An interrupted delivery can repeat after its lease expires. Confirm that downstream connectors reject duplicates or tolerate repeated requests. Then compare sent and delivered totals across the test.
- [] **Record the measured window** and, if needed, tune `heartbeat < fence < ttl` down proportionally and re-drill.

14.4 HA go-live checklist

- [] Server-DB backend. **DB-tier HA configured and its own failover rehearsed.**
- [] Identical engine version + config dir + `[store]` target on every node. `pool_size ≥ 3`. **clocks NTP-synced.**
- [] `/cluster/nodes` shows all nodes and exactly one `leader_node_id`.
- [] **VIP per inbound port** with a **TCP-connect health check**. Partner **reconnect-on-drop** verified.
- [] **Clean-switchover drill** passed ($\approx$ one heartbeat, VIP follows, zero loss).
- [] **Crash drill** passed (failover within the TTL, no double-processing, in-flight rows recovered, zero loss).
- [] Measured failover window documented. Lease timings tuned to your network.
- [] Apply configuration changes with a coordinated, non-rolling restart. Restart all nodes together to prevent different graphs during the change.
- [] `/cluster/status` + `/cluster/nodes` monitored (alert on **no live leader** beyond a failover window) alongside the §12 surfaces.

For throughput (not failover), scale intra-node: Each outbound connection has an independent delivery worker. A slow or failed connection does not block other connections. Use finite retries where a blocked FIFO queue would limit throughput. Adding cluster nodes does **not** raise throughput — only the leader processes.

15. Getting help & reporting bugs

- **Bugs & feature requests:** open a GitHub issue using the repository's issue templates (`bug_report.md` / `feature_request.md`).
- **Security vulnerabilities:** use the repository's **private security advisory** process per `.github/SECURITY.md` — do **not** open a public issue for a vulnerability.
- **Before filing:** verify against current `main` (pre-1.0, latest-main-only support), and include the engine version, config shape, and relevant **non-PHI** log excerpts.
-  **Never attach real PHI** to an issue, log excerpt, or reproduction. Reproduce with a synthetic corpus from `messagefoundry generate` .

16. Decommissioning a pilot

Decommissioning includes **disposing of PHI**. `uninstall-service.ps1` removes the service but **leaves the store and logs on disk**. Follow these steps:

1. **Graceful drain + stop**, confirm no in-flight work remains.
2. **Uninstall the service** (`scripts\service\uninstall-service.ps1`).
3. **Securely dispose of all PHI-bearing artifacts:** Remove the store (`.db` , `-wal` , and `-shm`) and PostgreSQL databases or backups. Remove File-connector spill directories and `logs` . Remove every backup copy and the encryption key or DPAPI key file.
4. **Revoke credentials** (service account, AD bind account, any API tokens).

Dispose of backup copies and encryption keys under the same rules as the live store. An encrypted backup remains recoverable when its key survives.

Appendix — where each topic is documented

Topic	Reference
Install + your config repo (consumer model)	INSTALL-GUIDE.md, ADR 0017
System requirements / sizing by volume	SYSTEM-REQUIREMENTS.md
Install + Windows service	SERVICE.md
Network exposure / TLS	DEPLOYMENT.md
High availability / clustering	CLUSTERING.md
Throughput baseline / tuning	TUNING-BASELINE.md
Service settings / environments	CONFIGURATION.md

Topic	Reference
Connections / the graph / connections.toml	CONNECTIONS.md, ADR 0007
Reliability / staged pipeline	ADR 0001
Security / auth / RBAC / audit	SECURITY.md
PHI handling / encryption	PHI.md
HL7 validation	HL7-VALIDATION.md
Load testing	LOAD-TESTING.md
Write-amplification / sizing	step-b-write-amplification.md
Built-vs-roadmap (authoritative)	ARCHITECTURE.md

© 2026 **MessageFoundry, LLC** · MessageFoundry is licensed under **AGPL-3.0-or-later** · Document generated September 2026 for MessageFoundry v0.3.2.

MessageFoundry and the MessageFoundry word mark are trademarks of MessageFoundry, LLC. This document describes an **Early Access** release; check pypi.org/project/messagefoundry for the current version. Verify specifics against your own deployment and the engine's reference documentation.